1. Como o Projeto Funciona do Começo ao Fim

Arquitetura em 2 Partes Desacopladas: Compilador Ascendente SLR(1) + Máquina Hipotética

Pipeline Completo de Compilação e Execução

```
// ================ PARTE 1: COMPILADOR ================
exemplos/correto.java.txt   (Código-Fonte em lalg-java)
   |
   ▼  [Fase 1] Lexer.java (AFD Manual Puro)
Lista de 87 Tokens <Tipo, Lexema, Linha, Coluna>
   |
   ▼  [Fase 2] ParserSLR.java + TabelaSLR.java
Árvore Sintática Concreta (CST - 95 Estados SLR(1))
   |
   ▼  [Fase 3] AnalisadorSemantico.java
Tabela de Símbolos Validada (Endereços 0, 1, 2, 3)
   |
   ▼  [Fase 4] TradutorCodigo.java (Com Backpatching)
codigo.objeto.txt (36 Instruções de Pilha)

// ============= PARTE 2: MÁQUINA HIPOTÉTICA =============
   ▼  [Fase 5] MaquinaHipotetica.java + MainMaquina.java
Execução na CPU Virtual (Registradores i, s e Pilha M[])
```

Por Que Essa Arquitetura Existe?

Zero Bibliotecas Externas
Proibido usar JFlex, Yacc ou ANTLR. Todo o compilador foi escrito em Java puro.

Desacoplamento Total
A Parte 1 gera texto puro (codigo.objeto.txt). A Parte 2 lê esse texto e executa na VM.

8 Arquivos Java Modulares
6 classes no pacote 'compilador' e 2 classes no pacote 'compilador.maquina'.

Modos de Depuração
Flag --debug mostra FIRST/FOLLOW e passos Shift/Reduce; flag --trace mostra a pilha da VM.

Pergunta Clássica da Banca
"Como a Parte 1 conversa com a Parte 2?" -> Pelo arquivo texto codigo.objeto.txt!

2. A Gramática: Cabeçalho PROG e Declarações DC / VAR

Entendendo as regras R0 a R7 do arquivo lalg-java.txt e o papel da recursão à direita

Regras R0 a R7 (Estrutura e Declarações)

```
R0: START    -> PROG   // Gramática Aumentada (S' -> S)
R1: PROG     -> public class id {
                  public static void main ( String [ ] id ) {
                    <CMDS>
                  }
                }

R2: DC       -> <VAR> <MAIS_CMDS>
R3: VAR      -> <TIPO> <VARS>
R4: VARS     -> id <MAIS_VAR>
R5: MAIS_VAR -> , <VARS>    // Recursão à direita p/ lista
R6: MAIS_VAR -> λ           // Fim da lista de variáveis
R7: TIPO     -> double      // Único tipo da linguagem
```

Por Que Cada Regra Foi Escrita Assim?

Por que existem 2 'id' em PROG?
O 1º id é o nome da classe (ex: Teste) e o 2º id é o argumento do main (ex: args).

Por que DC tem <MAIS_CMDS>?
Como MAIS_CMDS começa com ';', isso obriga toda declaração de variável a terminar com ';'.

Por que VAR separa TIPO de VARS?
Para que o semântico saiba o tipo ('double') antes de cadastrar a lista de nomes na tabela.

Por que MAIS_VAR deriva λ (vazio)?
O λ permite encerrar a declaração quando não há mais vírgulas antes do ponto-e-vírgula.

Posição de <CMDS> em PROG
Contando de 0 a 17 nos símbolos de PROG, <CMDS> está exatamente no índice 15!

3. O Tipo 'double' e o Segredo da Regra <CMDS>

Por que só existe double e como a gramática diferencia comandos com ';' de blocos '{ }'

Regras R7 a R12 (Tipo e Fluxo de Comandos)

```
R7:  TIPO      -> double

// AS 4 ALTERNATIVAS DE <CMDS>:
R8:  CMDS      -> <CMD> <MAIS_CMDS>      // Exige ';' após CMD
R9:  CMDS      -> <CMD_COND> <CMDS>      // Sem ';' após '}'
R10: CMDS      -> <DC>                   // Declaração (exige ';')
R11: CMDS      -> λ                      // Fim de bloco '}'

R12: MAIS_CMDS -> ; <CMDS>               // O ';' mora aqui!

// Exemplo válido aceito por CMDS:
double cont;          // via R10 (DC -> VAR ; CMDS)
cont = 10;            // via R8  (CMD ; CMDS)
while (cont > 0) { }  // via R9  (CMD_COND CMDS - sem ';')
```

Explicação para a Defesa Oral

Por que só existe 'double'?
Simplifica a VM: cada variável ocupa 1 índice no array double[] M = new double[10000].

O Segredo do Ponto-e-Vírgula
MAIS_CMDS -> ; <CMDS> garante que atribuições, prints e declarações exijam ';'.

Por que if/while não têm ';'?
Em R9 (CMDS -> CMD_COND CMDS), após fechar '}' do if/while, vem CMDS direto sem ';'.

Para que serve R11 (CMDS -> λ)?
Permite que qualquer bloco '{ <CMDS> }' termine quando o parser encontra '}'.

Declarações no meio do código
Como CMDS -> DC e DC -> VAR ; CMDS, podemos declarar variáveis em qualquer ponto do bloco.

4. Comandos, Condições, lerDouble() e Dangling Else

Regras R13 a R28: Por que lerDouble() fica em EXP_IDENT e por que não há ambiguidade no else

Regras R13 a R28 (If/While, Atribuição e Condição)

```
R13: CMD_COND    -> if ( <CONDICAO> ) { <CMDS> } <PFALSA>
R14: CMD_COND    -> while ( <CONDICAO> ) { <CMDS> }
R15: CMD         -> System.out.println ( <EXPRESSAO> )
R16: CMD         -> id <RESTO_IDENT>
R17: PFALSA      -> else { <CMDS> }
R18: PFALSA      -> λ
R19: RESTO_IDENT -> = <EXP_IDENT>
R20: EXP_IDENT   -> <EXPRESSAO>
R21: EXP_IDENT   -> lerDouble()   // <-- SÓ PODE EM ATRIBUIÇÃO!
R22: CONDICAO    -> <EXPRESSAO> <RELACAO> <EXPRESSAO>
R23..R28: RELACAO -> == | != | >= | <= | > | <
```

Respostas Críticas sobre R13 a R28

Sem Conflito Dangling Else

Chaves '{ <CMDS> }' são obrigatórias no if e no else. O '}' delimita o bloco sem ambiguidade.

Restrição do lerDouble()

Fica em EXP_IDENT (R21) para permitir 'a = lerDouble();' e proibir 'a = lerDouble() + 1;'.

System.out.println é 1 Token

A gramática não tem objetos nem operador ponto '.'; o comando de saída é um terminal único.

Fatoração em CMD (R16 + R19)

id seguido de RESTO_IDENT (= EXP_IDENT) isola o identificador alvo da atribuição.

6 Operadores Relacionais

Cada operador (==, !=, >=, <=, >, <) mapeia 1-para-1 para uma instrução da Máquina Virtual.

5. Hierarquia de Expressões, Precedência e Unário

Regras R29 a R43: Como EXPRESSAO, TERMO e FATOR garantem a prioridade matemática correta

Regras R29 a R43 (Aritmética em 3 Níveis)

```
// NÍVEL 1 (Menor Precedência): Soma (+) e Subtração (-)
R29: EXPRESSAO     -> <TERMO> <OUTROS_TERMOS>
R36: OUTROS_TERMOS -> <OP_AD> <TERMO> <OUTROS_TERMOS>
R37: OUTROS_TERMOS -> λ
R38: OP_AD -> +    |   R39: OP_AD -> -

// NÍVEL 2 (Média Precedência): Unário (-), Mult (*) e Div (/)
R30: TERMO         -> <OP_UN> <FATOR> <MAIS_FATORES>
R31: OP_UN -> -    |   R32: OP_UN -> λ
R40: MAIS_FATORES  -> <OP_MUL> <FATOR> <MAIS_FATORES>
R41: MAIS_FATORES  -> λ
R42: OP_MUL -> *   |   R43: OP_MUL -> /

// NÍVEL 3 (Maior Precedência): Variável, Número e Parênteses
R33: FATOR -> id   | R34: FATOR -> numero_real | R35: ( <EXPRESSAO> )
```

Por Que Essa Modelagem É Perfeita?

Precedência Automática
Em '2 + 3 * 4', '3 * 4' é reduzido primeiro dentro de TERMO antes da SOMA em EXPRESSAO.

Parênteses Quebram Precedência
R35 FATOR -> (EXPRESSAO) reinicia a hierarquia no nível mais profundo.

Operador Unário (OP_UN)
Trata sinal negativo unário (-x ou -10) separado da subtração binária (OP_AD).

Recursão à Direita
OUTROS_TERMOS e MAIS_FATORES usam recursão à direita com λ, evitando conflitos na tabela.

Tradução Pós-Fixa (RPN)
Ao percorrer essa árvore, os operandos são empilhados antes do operador automaticamente.

6. O Tipo do Compilador: Ascendente SLR(1) (Bottom-Up)

Como funciona a técnica Shift-Reduce e por que ela difere de um parser Descendente (LL)

Funcionamento do Motor Shift-Reduce SLR(1)

```
pilhaDeEstados : [ 0, s_1, s_2, ..., s_k ]
pilhaDeNos     : [    No_1, No_2, ..., No_k ]
Entrada (Fita) : [ token_atual, proximo_token, ..., $ (EOF) ]

A cada passo, olhamos:
  • estadoAtual = Estado inteiro no topo de pilhaDeEstados
  • tokenAtual  = Token atual na entrada (lookahead de 1 símbolo)

Consultamos ACTION[estadoAtual, tokenAtual]:
  1. SHIFT(p)    -> Empilha tokenAtual em pilhaDeNos e estado 'p' em pilhaDeEstados
  2. REDUCE(R)   -> Desempilha |ladoDireito| itens de cada pilha,
                    consulta proximoEstado = GOTO[topo, ladoEsquerdo] e empilha
  3. ACCEPT      -> Reduziu PROG completo e leu '$' -> SUCESSO!
  4. ERROR       -> Célula vazia -> Lança SyntacticException
```

O Que Significa a Sigla SLR(1)?

S = Simple (Simples)
Usa os estados canônicos LR(0) combinados com o conjunto FOLLOW para decidir reduções.

L = Left-to-right
Lê a fita de tokens da esquerda para a direita, um token por vez.

R = Rightmost derivation
Reconhece uma derivação mais à direita ao contrário (em ordem reversa, de baixo para cima).

(1) = 1 Token de Lookahead
Toma todas as decisões olhando apenas 1 token à frente na fita de entrada.

Validação Formal da Gramática
95 estados LR(0), 0 conflitos Shift/Reduce e 0 conflitos Reduce/Reduce!

7. Conjuntos FIRST: O Que São e Como Foram Calculados

Entendendo quais terminais podem iniciar cada Não-Terminal e o efeito das regras vazias (λ)

Cálculo de FIRST na Gramática lalg-java

```
// Exemplos Diretos:
FIRST(PROG)       = { PUBLIC }
FIRST(DC)         = { DOUBLE }   (vem de VAR -> TIPO -> double)
FIRST(CMD_COND)   = { IF, WHILE }
FIRST(CMD)        = { PRINT, ID }
FIRST(FATOR)      = { ABRE_PAR '(', NUMERO_REAL, ID }

// Exemplos com Propagação de λ (Anuláveis):
// Como TERMO -> OP_UN FATOR MAIS_FATORES e OP_UN -> - | λ:
FIRST(OP_UN)      = { MENOS '-' }  (e anulável λ)
FIRST(TERMO)      = FIRST(OP_UN) ⍰ FIRST(FATOR)
                  = { ABRE_PAR '(', MENOS '-', NUMERO_REAL, ID }

// Como CMDS -> CMD.. | CMD_COND.. | DC | λ:
FIRST(CMDS)       = { DOUBLE, IF, WHILE, PRINT, ID } (e λ)
```

Os 6 Não-Terminais Anuláveis (Geram λ)

1. MAIS_VAR (R6)
Permite terminar declaração sem mais vírgulas. FIRST = { ',' } + λ.

2. CMDS (R11)
Permite bloco '{ }' vazio ou fim de comandos antes de '}'. FIRST = { double, if, while, print, id } + λ.

3. PFALSA (R18)
Permite comando 'if' sem bloco 'else'. FIRST = { else } + λ.

4. OP_UN (R32)
Permite fator positivo sem sinal '-' na frente. FIRST = { '-' } + λ.

5 e 6. OUTROS_TERMOS e MAIS_FATORES
Permitem encerrar somas (R37) e multiplicações (R41) quando não há mais operadores.

8. Conjuntos FOLLOW: O Coração das Reduções SLR(1)

Por que o FOLLOW existe e como ele elimina conflitos nas reduções da Tabela ACTION

Cálculo de FOLLOW e seu Papel no SLR(1)

```
// De onde vem cada FOLLOW principal?
FOLLOW(START / PROG) = { EOF '$' }         // Fim do arquivo
FOLLOW(CMDS)         = { FECHA_CHAVE '}' } // Blocos { <CMDS> }
FOLLOW(DC)           = { FECHA_CHAVE '}' } // CMDS -> DC no fim
FOLLOW(VAR / VARS)   = { PONTO_VIRG ';' }  // Seguidos por MAIS_CMDS
FOLLOW(CMD)          = { PONTO_VIRG ';' }  // CMDS -> CMD ; CMDS
FOLLOW(CONDICAO)     = { FECHA_PAR ')' }   // if ( <CONDICAO> )
FOLLOW(CMD_COND)     = { DOUBLE, IF, WHILE, PRINT, ID, '}' }
FOLLOW(PFALSA)       = { DOUBLE, IF, WHILE, PRINT, ID, '}' }

// No código de TabelaSLR.java (linhas 107-113):
private void adicionarReduce(int estadoAtual, int numeroDaRegra) {
    Regra regra = regrasDaGramatica.get(numeroDaRegra);
    for (Lexer.Tipo tokenDoFollow : conjuntoFollow.get(regra.ladoEsquerdo)) {
        tabelaAction.computeIfAbsent(estadoAtual, ...).put(tokenDoFollow, ...);
    }
}
```

Como o FOLLOW Resolve Conflitos (LR(0) vs SLR(1))

O Problema do LR(0) Puro

No Estado 16 (início do main), temos 'CMDS -> .' (vazio) e shifts para 'double, if, while...'. Um LR(0) reduziria em qualquer token, gerando conflito!

A Solução do SLR(1)

Como FOLLOW(CMDS) = { '}' }, o Estado 16 só faz REDUCE(CMDS -> λ) se o token atual for '}'!

Outro Exemplo: Estado 35 (VARS)

Após ler 'double a', temos 'MAIS_VAR -> .'. Se vier ',', faz SHIFT(51); se vier ';' (que está em FOLLOW(MAIS_VAR)), faz REDUCE(R6)!

Resultado Matemático

Interseção entre tokens de SHIFT e conjuntos FOLLOW das reduções é vazia em todos os 95 estados!

9. Estados 0 a 15: O Que É um Estado e o Cabeçalho PROG

Entendendo os Itens LR(0) [A -> α . β] e a sequência de abertura da classe e do método main

Mapa dos Estados 0 a 15 (Cabeçalho do Programa)

```
Estado 0  --[SHIFT PUBLIC]--> Estado 1
Estado 0  --[GOTO PROG]-----> Estado 2  --> [ACCEPT em EOF '$']

Estado 1  --[SHIFT CLASS]---> Estado 3  ([PROG -> public class . id ...])
Estado 3  --[SHIFT ID]------> Estado 4  ([PROG -> public class id . { ...])
Estado 4  --[SHIFT '{']-----> Estado 5
Estado 5  --[SHIFT PUBLIC]--> Estado 6
Estado 6  --[SHIFT STATIC]--> Estado 7
Estado 7  --[SHIFT VOID]----> Estado 8
Estado 8  --[SHIFT MAIN]----> Estado 9
Estado 9  --[SHIFT '(']-----> Estado 10
Estado 10 --[SHIFT STRING]--> Estado 11
Estado 11 --[SHIFT '[']-----> Estado 12
Estado 12 --[SHIFT ']']-----> Estado 13
Estado 13 --[SHIFT ID]------> Estado 14 (nome do parâmetro 'args')
Estado 14 --[SHIFT ')']-----> Estado 15
Estado 15 --[SHIFT '{']-----> Estado 16 (Entrou no corpo do main!)
```

Entendendo os Estados 0 a 15

Por que Estado 2 é especial?

O Estado 2 representa o item [START -> PROG .]. Quando o próximo token é EOF ($), dispara ACCEPT!

Caminho Linear (1 -> 3..16)

Como o cabeçalho de Java é fixo, os estados 1 e 3 a 15 têm apenas 1 transição de SHIFT cada.

E os fechamentos de PROG?

No final do programa, após reduzir <CMDS> no main, o parser vai ao Estado 25 ('}' do main -> 37) e Estado 37 ('}' da classe -> 53, que reduz R1: PROG).

Dica para Defesa Oral

Se o professor perguntar 'Por que pula do 1 para o 3?', é porque o Estado 2 foi reservado para GOTO(0, PROG)!

10. Os 6 'Hubs de Comandos': Estados 16, 26, 33, 73, 81 e 92

Por que existem 6 estados quase idênticos em TabelaSLR.java e qual a diferença entre eles

Anatomia dos 6 Estados 'Hubs de <CMDS>'

```
// Em TabelaSLR.java (configurarHubDeComandos - linhas 245-259):
adicionarShift(estadoHub, DOUBLE, 17); // Inicia declaração: double ...
adicionarShift(estadoHub, IF,     18); // Inicia condicional: if (...) ...
adicionarShift(estadoHub, WHILE,  19); // Inicia repetição: while (...) ...
adicionarShift(estadoHub, PRINT,  20); // Inicia saída: System.out.println(...)
adicionarShift(estadoHub, ID,     21); // Inicia atribuição: id = ...
adicionarReduce(estadoHub, 11);        // Em '}': reduz R11 (CMDS -> vazio)

// O QUE MUDA ENTRE ELES? O destino GOTO(estadoHub, CMDS)!
• Estado 16 (Corpo do main)         --[GOTO CMDS]--> Estado 25 (espera '}' do main)
• Estado 26 (Após um if ou while)   --[GOTO CMDS]--> Estado 38 (reduz R9: CMD_COND CMDS)
• Estado 33 (Após ponto-e-vírgula) --[GOTO CMDS]--> Estado 50 (reduz R12: ; CMDS)
• Estado 73 (Corpo do bloco if)     --[GOTO CMDS]--> Estado 82 (espera '}' do if)
• Estado 81 (Corpo do bloco while) --[GOTO CMDS]--> Estado 86 (espera '}' do while)
• Estado 92 (Corpo do bloco else)   --[GOTO CMDS]--> Estado 93 (espera '}' do else)
```

Por Que Eles Não Podem Ser Fundidos em 1 Só?

Memória do Contexto na Pilha

O autômato LR precisa saber QUEM abriu aquele bloco de comandos para reduzir a regra certa no final!

Exemplo no 'while' (Estado 81)

Quando o corpo do while termina em '}', o Estado 81 reduz CMDS e vai para o Estado 86, que consome '}' e reduz R14 (while).

Exemplo no 'if' (Estado 73)

Quando o corpo do if termina, o Estado 73 reduz CMDS e vai para o Estado 82 -> 87, que verifica se existe 'else'!

Domínio Total na Defesa

Saber explicar esses 6 estados impressiona qualquer banca de Compiladores!

11. Estados de Declaração, Atribuição e Impressão

Como o autômato processa 'double a, b, c;', 'x = lerDouble();' e 'System.out.println(c);'

Fluxo de Estados: Declarações e Comandos Simples

```
// 1. DECLARAÇÃO: double a, b;
Hub(16) --[DOUBLE]--> 17 (Reduce R7: TIPO -> double)
16 --[GOTO TIPO]--> 24 --[SHIFT ID 'a']--> Estado 35
Estado 35: Tem vírgula?
  • SIM (','): SHIFT 51 --[ID 'b']--> 35 (Loop de variáveis!)
  • NÃO (';'): REDUCE R6 (MAIS_VAR -> λ) -> 52 (Reduce R4: VARS)
               -> 72 (Reduce R5) -> 36 (Reduce R3: VAR) -> 23
Estado 23 --[SHIFT ';']--> 33 (Hub após ';') -> 50 -> 34 (R2: DC)

// 2. ATRIBUIÇÃO: a = lerDouble();  ou  c = a - b;
Hub --[ID]--> 21 --['=']--> Estado 31
  • Se LER_DOUBLE: SHIFT 47 (Reduce R21: EXP_IDENT -> lerDouble())
  • Se EXPRESSAO : GOTO 49 (Reduce R20: EXP_IDENT -> EXPRESSAO)
  -> GOTO 48 (Reduce R19: RESTO_IDENT) -> 32 (Reduce R16: CMD)

// 3. SAÍDA: System.out.println(c);
Hub --[PRINT]--> 20 --['(']--> 30 --[GOTO EXPRESSAO]--> 46 --[')']--> 71 (R15)
```

Pontos-Chave Deste Grupo de Estados

Estado 35 (Loop da Vírgula)
Tem SHIFT para ',' (vai ao 51) e REDUCE(R6) para ';' (que é o FOLLOW de MAIS_VAR).

Estado 31 (Direita do '=')
É o único estado da tabela que aceita o token LER_DOUBLE (shift 47), pois lerDouble() só é válido após '='.

Estados 23 e 27 (Espera de ';')
O Estado 23 espera ';' após uma declaração VAR; o Estado 27 espera ';' após um comando simples CMD.

Reconstrução Bottom-Up
Em 'double a, b, c;', o parser empilha tudo e reduz c -> b,c -> a,b,c da direita para a esquerda!

12. Estados de Expressões, Condições e o Estado 87 (Else)

Como o autômato resolve o operador unário (-), os relacionais (Estado 42) e o bloco if/else

Mapa de Estados: Expressões, Relacionais e If/Else

```
// 1. INÍCIO DE EXPRESSÃO E OPERADOR UNÁRIO (Estados 28..31, 61, 65, 68):
• Se vier '-'                --> SHIFT 40 (Reduce R31: OP_UN -> -)
• Se vier ID, NUM ou '('     --> REDUCE R32 (OP_UN -> λ)
• Após OP_UN (Estado 44/80) --> SHIFT ID(66), NUM(67), '('(68)

// 2. PRECEDÊNCIA DE * E / (Estados 69 e 85) vs + E - (Estados 43 e 75):
• Estado 69/85: SHIFT '*'(77) ou '/'(78) | REDUCE R41 (MAIS_FATORES -> λ)
• Estado 43/75: SHIFT '+'(62) ou '-'(63) | REDUCE R37 (OUTROS_TERMOS -> λ)

// 3. OPERADORES RELACIONAIS (Estado 42):
• SHIFT '=='(55), '!='(56), '>='(57), '<='(58), '>'(59), '<'(60)
• GOTO(42, RELACAO) -> Estado 61 (lê 2ª EXPRESSAO) -> Estado 74 (R22)

// 4. O DECISOR DO IF/ELSE (Estado 87 - logo após '}' do if):
• Se vier ELSE               --> SHIFT 90 -> 92('{') -> 93 -> 94 (R17)
• Se vier FOLLOW(PFALSA)     --> REDUCE R18 (PFALSA -> λ) -> 91 (R13: if)
```

Resumo Completo dos 95 Estados (0 a 94)

Estados 0..15, 25, 37, 53
Cabeçalho e fechamento da classe e do método main (Regra R1: PROG).

Estados 16, 26, 33, 73, 81, 92
Os 6 Hubs de <CMDS> (início de main, pós-bloco, pós-';', if, while, else).

Estados 17, 22..24, 34..36, 51, 52, 72
Declaração de variáveis 'double id, id;' (Regras R2 a R7).

Estados 20, 21, 27, 30..32, 46..49, 71
Comandos simples: atribuição '=', lerDouble() e System.out.println.

Estados 18, 19, 28, 29, 40..45, 54..70, 74..94
Expressões aritméticas, condições relacionais e blocos if/else/while.

13. Arquivo 1: Lexer.java (Estrutura, Tokens e Exceções)

Qual a parte dele, por que existe e como organiza os 34 tipos de tokens da linguagem

Dentro de src/compilador/Lexer.java (Linhas 16-105)

```
public class Lexer {
    public enum Tipo {
        PUBLIC, CLASS, STATIC, VOID, MAIN, STRING, DOUBLE, IF, ELSE, WHILE,
        PRINT, LER_DOUBLE, ID, NUMERO_REAL,
        ABRE_PAR, FECHA_PAR, ABRE_CHAVE, FECHA_CHAVE, ABRE_COLCH, FECHA_COLCH,
        PONTO_VIRG, VIRGULA, ATRIB, EQ, NE, GE, LE, GT, LT,
        MAIS, MENOS, MULT, DIV, EOF
    }

    public static class Token {
        public final Tipo tipo;
        public final String lexema;
        public final int linha, coluna;
    }

    private static final Map<String, Tipo> PALAVRAS_RESERVADAS = Map.of(
        "public", Tipo.PUBLIC, "class", Tipo.CLASS, "static", Tipo.STATIC,
        "void", Tipo.VOID, "main", Tipo.MAIN, "String", Tipo.STRING,
        "double", Tipo.DOUBLE, "if", Tipo.IF, "else", Tipo.ELSE, "while", Tipo.WHILE
    );
    private final String codigoFonte;
    private int posicaoAtual = 0, linhaAtual = 1, colunaAtual = 1;
```

O Papel de Cada Componente do Lexer

enum Tipo (Linhas 21-42)
Representa o vocabulário terminal formal da gramática. Usado como chave nas tabelas ACTION e FIRST/FOLLOW.

class Token (Linhas 47-64)
Guarda linha e coluna do início do token para exibir mensagens de erro precisas em qualquer fase.

class LexicalException (69-77)
Exceção disparada quando encontra caractere inválido (ex: '@') ou comentário '/*' não fechado.

PALAVRAS_RESERVADAS (Linhas 80-91)
Usa Map.of(...) para diferenciar uma palavra-chave (como 'double' ou 'if') de um nome de variável comum em O(1).

fromFile() (Linhas 102-104)
Usa Files.readString(Path.of(caminhoArquivo), StandardCharsets.UTF_8) para carregar o código-fonte.

14. Arquivo 1: Lexer.java (O Motor AFD Linha por Linha)

Como funcionam proximoToken(), pularEspacosEComentarios(), consumirSeProximoFor() e casos especiais

Linhas Críticas de Lexer.java (Linhas 122-218)

```
public Token proximoToken() {
    pularEspacosEComentarios();
    if (posicaoAtual >= codigoFonte.length()) return new Token(Tipo.EOF, "$", linhaAtual, colunaAtu
    int linhaToken = linhaAtual, colunaToken = colunaAtual;
    char caractere = lerProximoCaractere();
    if (Character.isLetter(caractere) || caractere == '_') {
        int inicio = posicaoAtual - 1;
        while (posicaoAtual < codigoFonte.length() && (Character.isLetterOrDigit(...) || ...))
            lerProximoCaractere();
        String palavra = codigoFonte.substring(inicio, posicaoAtual);
        if ("System".equals(palavra) && codigoFonte.startsWith(".out.println", posicaoAtual)) {
            for (int i = 0; i < ".out.println".length(); i++) lerProximoCaractere();
            return new Token(Tipo.PRINT, "System.out.println", linhaToken, colunaToken);
        }
        if ("lerDouble".equals(palavra) && codigoFonte.startsWith("()", posicaoAtual)) {
            lerProximoCaractere(); lerProximoCaractere();
            return new Token(Tipo.LER_DOUBLE, "lerDouble()", linhaToken, colunaToken);
        }
        return new Token(PALAVRAS_RESERVADAS.getOrDefault(palavra, Tipo.ID), palavra, ...);
    }
    // No switch de operadores (Linhas 191-193):
    case '=': return consumirSeProximoFor('=') ? new Token(Tipo.EQ, "==", ...) : new Token(Tipo.ATF
```

O Que o Professor Pode Perguntar Aqui?

"Por que o for consome '.out.println'.length()?"
Porque a string ".out.println" tem 12 caracteres, mantendo o contador colunaAtual sempre exato!

"Por que usar PALAVRAS_RESERVADAS.getOrDefault?"
Na linha 155: se a palavra estiver no mapa (ex: 'if'), retorna Tipo.IF; senão, retorna o padrão Tipo.ID.

"Como funciona consumirSeProximoFor(char esperado)?"
Se o próximo caractere for igual ao esperado (ex: segundo '=' em '=='), consome e retorna true; senão retorna false.

"E se tiver um '!' sozinho no código?"
Na linha 201: se após '!' não vier '=', consumirSeProximoFor('=') dá false e lança LexicalException!

15. Arquivo 2: TabelaSLR.java (Gramática, ACTION e GOTO)

Como as 44 regras, os conjuntos FIRST/FOLLOW, os 5 padrões e os 95 estados são armazenados

Dentro de src/compilador/TabelaSLR.java (Linhas 20-113)

```
public class TabelaSLR {
    public static class Regra {
        public final int numeroRegra;
        public final ParserSLR.NaoTerminal ladoEsquerdo; // Antes da seta (->)
        public final Object[] ladoDireito;               // Depois da seta (->)
    }
    public enum AcaoTipo { SHIFT, REDUCE, ACCEPT, ERROR }

    private final List<Regra> regrasDaGramatica = new ArrayList<>();
    private final Map<NaoTerminal, Set<Lexer.Tipo>> conjuntoFirst  = new EnumMap<>(...);
    private final Map<NaoTerminal, Set<Lexer.Tipo>> conjuntoFollow = new EnumMap<>(...);
    private final Map<Integer, Map<Lexer.Tipo, Acao>> tabelaAction = new HashMap<>();
    private final Map<Integer, Map<NaoTerminal, Integer>> tabelaGoto = new HashMap<>();

    // A FUNÇÃO MAIS IMPORTANTE DO SLR(1) (Linhas 107-113):
    private void adicionarReduce(int estadoAtual, int numeroDaRegra) {
        Regra regra = regrasDaGramatica.get(numeroDaRegra);
        for (Lexer.Tipo tokenDoFollow : conjuntoFollow.get(regra.ladoEsquerdo)) {
            tabelaAction.computeIfAbsent(estadoAtual, k -> new HashMap<>())
                        .put(tokenDoFollow, new Acao(AcaoTipo.REDUCE, 0, regra));
        }
    }
```

Decisões Técnicas de TabelaSLR.java

Por que ladoDireito é Object[]?
Porque o lado direito de uma regra mistura Terminais (Lexer.Tipo) e Não-Terminais (ParserSLR.NaoTerminal).

Regras vazias têm ladoDireito.length == 0
Exemplo: adicionarRegra(CMDS) cria um array ladoDireito vazio de tamanho 0. No reduce, desempilha 0 itens!

5 Métodos de Padrões Reutilizáveis
configurarHubDeComandos, configurarInicioExpressao, configurarInicioFator, configurarContinuacaoSoma e Mult evitam repetição!

obterTokensEsperados(int estadoAtual)
Na linha 70: retorna tabelaAction.get(estadoAtual).keySet(), listando quais tokens eram esperados no erro!

16. Arquivo 3: ParserSLR.java (O Motor Shift-Reduce)

Explicação linha por linha das duas pilhas sincronizadas (pilhaDeEstados e pilhaDeNos) e da CST

Dentro de src/compilador/ParserSLR.java (Linhas 93-138)

```
Deque<Integer> pilhaDeEstados = new ArrayDeque<>();
Deque<NoSintatico> pilhaDeNos = new ArrayDeque<>();
pilhaDeEstados.push(0); // Começa no Estado 0
int indiceTokenAtual = 0;
while (true) {
    int estadoAtual = pilhaDeEstados.peek();
    Lexer.Token tokenAtual = tokens.get(indiceTokenAtual);
    TabelaSLR.Acao acao = tabela.obterAcao(estadoAtual, tokenAtual.tipo);
    switch (acao.tipo) {
        case SHIFT:
            pilhaDeNos.push(new NoSintatico(tokenAtual)); // 1. Empilha folha (token)
            pilhaDeEstados.push(acao.estadoDestino);      // 2. Empilha estado destino
            indiceTokenAtual++;                           // 3. Avança para próximo token
            break;
        case REDUCE:
            TabelaSLR.Regra regra = acao.regraParaReduzir;
            NoSintatico novoNoPai = new NoSintatico(regra.ladoEsquerdo);
            for (int k = 0; k < regra.ladoDireito.length; k++) {
                pilhaDeEstados.pop();
                novoNoPai.filhos.add(0, pilhaDeNos.pop()); // Mantém ordem esq->dir
            }
            int proximoEstado = tabela.obterGoto(pilhaDeEstados.peek(), regra.ladoEsquerdo);
            pilhaDeNos.push(novoNoPai);       // Empilha não-terminal reduzido
```

As 3 Perguntas Que o Professor Faz Aqui

1. "Por que usar duas pilhas sincronizadas?"
Separar pilhaDeEstados (Integer) e pilhaDeNos (NoSintatico) elimina casts de Object e deixa claro que cada nó tem um estado!

2. "Por que usa novoNoPai.filhos.add(0, ...)?"
Porque a pilha é LIFO (desempilha da direita para a esquerda). Inserindo sempre no índice 0, os filhos ficam na ordem da esquerda para a direita!

3. "O que acontece em uma regra vazia (λ)?"
Em regras vazias (como CMDS -> λ), regra.ladoDireito.length é 0. O for não roda (desempilha 0 itens) e o parser apenas empilha o nó vazio e o estado GOTO!

17. Arquivo 4: AnalisadorSemantico.java (Tabela de Símbolos)

Validação de declaração prévia, duplicidade e mapeamento de endereços 0..n para a Máquina Virtual

Dentro de src/compilador/AnalisadorSemantico.java

```
public static class TabelaSimbolos {
    private final Map<String, Simbolo> variaveisDeclaradas = new LinkedHashMap<>();

    public Simbolo inserir(Lexer.Token tokenVariavel) {
        String nome = tokenVariavel.lexema;
        if (variaveisDeclaradas.containsKey(nome)) {
            throw new SemanticException("Variavel '" + nome + "' ja declarada...", ...);
        }
        Simbolo novo = new Simbolo(nome, "double", variaveisDeclaradas.size(), ...);
        variaveisDeclaradas.put(nome, novo);
        return novo;
    }
}
// Travessia Direta dos Comandos do main (Linhas 90-132):
public TabelaSimbolos analisar(ParserSLR.NoSintatico noRaizProg) {
    percorrerComandos(noRaizProg.getFilho(15)); // Filho 15 = <CMDS> do main
    return tabela;
}
private void percorrerComandos(ParserSLR.NoSintatico no) {
    if (no == null) return;
    if (no.naoTerminal == NaoTerminal.VAR) { cadastrarVariaveis(no); return; }
    if (no.getTipoToken() == Lexer.Tipo.ID) { tabela.verificarDeclarada(no.token); return; }
    for (NoSintatico filho : no.filhos) percorrerComandos(filho);
```

Por Que Funciona Assim?

Endereço = variaveisDeclaradas.size() (Linha 51)

A 1ª variável inserida pega endereço 0, a 2ª pega 1, a 3ª pega 2... mapeando direto para M[0], M[1], M[2] na VM!

Por que começar em getFilho(15)?

Porque os tokens 'Teste' (nome da classe) e 'args' (parâmetro) ficam no cabeçalho antes do índice 15. Dentro de <CMDS>, todo ID é variável!

percorrerComandos em Pré-Ordem

Visita os comandos na ordem das linhas do arquivo. Se usar uma variável na linha 3 e só declará-la na linha 8, dá erro!

cadastrarVariaveis (Linhas 124-132)

Percorre a subárvore de VAR inserindo todos os tokens ID ('double a, b, c;') na Tabela de Símbolos.

18. Arquivo 5: TradutorCodigo.java (Geração de Código Objeto)

Como a árvore sintática é traduzida para instruções de pilha (INPP, ALME, CRCT, CRVL, ARMZ, DESM, PARA)

Dentro de src/compilador/TradutorCodigo.java

```
public void traduzir(ParserSLR.NoSintatico noRaizProg) {
    emitir("INPP"); // INPP = Iniciar Programa Principal
    int totalVars = tabelaSimbolos.getTotalVariaveis();
    if (totalVars > 0) emitir("ALME", totalVars); // Reserva M[0..totalVars-1]

    traduzirBlocoComandos(noRaizProg);

    if (totalVars > 0) emitir("DESM", totalVars); // Libera as variáveis
    emitir("PARA");                               // Desliga a VM
}
private void traduzirBlocoComandos(ParserSLR.NoSintatico no) {
    if (no == null) return;
    if (no.naoTerminal == NaoTerminal.CMD)      { traduzirComandoSimples(no); return; }
    if (no.naoTerminal == NaoTerminal.CMD_COND) { traduzirComandoCondicionalOuLoop(no); return; }
    for (NoSintatico filho : no.filhos) traduzirBlocoComandos(filho);
}
// Tradução de TERMO e Operador Unário (Linhas 180-189):
private void traduzirTermo(ParserSLR.NoSintatico noTermo) {
    traduzirFator(noTermo.getFilho(1)); // 1º Empilha o FATOR
    if (noTermo.getFilho(0).getQtdFilhos() > 0) emitir("INVE"); // 2º Se tinha '-', inverte!
    traduzirMaisFatores(noTermo.getFilho(2));
}
```

Detalhes de Implementação do Tradutor

Busca Direta em traduzirBlocoComandos

Percorre os filhos ignorando nós estruturais (DC, MAIS_CMDS, ';') e dispara a tradução exatamente ao encontrar CMD ou CMD_COND!

Ordem no Operador Unário (INVE)

Na árvore, OP_UN é o filho(0) e FATOR é o filho(1). Mas na máquina de pilha precisamos empilhar o número PRIMEIRO (filho 1) para depois inverter seu sinal com INVE!

Formatação de Inteiros vs Reais

Em Instrucao.toString() (linha 29): 'if (argumento == argumento.longValue())' formata 4.0 como 'ALME 4' em vez de 'ALME 4.0'.

Despacho em traduzirComandoSimples

Se filho(0) for PRINT, traduz a expressão e emite IMPR; se for ID, emite LEIT (ou expressão) e ARMZ no endereço da variável.

19. Arquivo 5: TradutorCodigo.java (Backpatching no If e While)

Como o compilador preenche os endereços de salto DSVF e DSVI em uma única passada

O Código do Backpatching (Linhas 54-56 e 116-146)

```
public void corrigirDesvio(int linhaSalto, int linhaDestino) {
    listaInstrucoes.get(linhaSalto).argumento = (double) linhaDestino;
}

if (tipo == Lexer.Tipo.IF) {
    traduzirCondicao(noCondicao);
    int linhaDsvf = emitir("DSVF", -1);                  // 1. Emite DSVF com -1
    traduzirBlocoComandos(noCorpoPrincipal);             // 2. Traduz corpo do IF
    ParserSLR.NoSintatico noElse = noCmdCond.getFilho(7);
    if (noElse.getQtdFilhos() >= 4) {                    // Tem ELSE!
        int linhaDsvi = emitir("DSVI", -1);              // 3. Fim do IF pula o ELSE
        corrigirDesvio(linhaDsvf, listaInstrucoes.size()); // 4. DSVF aponta p/ início do ELSE
        traduzirBlocoComandos(noElse.getFilho(2));       // 5. Traduz corpo do ELSE
        corrigirDesvio(linhaDsvi, listaInstrucoes.size()); // 6. DSVI aponta p/ fim do ELSE
    } else {
        corrigirDesvio(linhaDsvf, listaInstrucoes.size()); // Sem ELSE: DSVF aponta p/ fim do IF
    }
} else if (tipo == Lexer.Tipo.WHILE) {
    int linhaInicioWhile = listaInstrucoes.size();       // Guarda linha do teste
    traduzirCondicao(noCondicao);
    int linhaDsvf = emitir("DSVF", -1);                  // Sai do while se falso
    traduzirBlocoComandos(noCorpoPrincipal);             // Corpo do while
    emitir("DSVI", linhaInicioWhile);                    // Volta para testar condição
```

Por Que Essa Técnica É Essencial?

Compilação em 1 Passada (One-Pass)
Graças ao Backpatching, não precisamos percorrer a árvore duas vezes para calcular o tamanho dos blocos.

Suporta Aninhamento Infinito
Como 'linhaDsvf' e 'linhaDsvi' são variáveis locais da chamada recursiva, podemos ter if dentro de while dentro de if sem conflito!

Por que DSVI no While não usa -1?
Porque o endereço de início do while (linhaInicioWhile = linha 4) fica para trás e JÁ É conhecido quando o while termina na linha 31!

Por que DSVF usa -1?
Porque o endereço de saída do while (linha 32) fica no futuro e só é descoberto após gerar o DSVI 4.

20. Arquivo 6: MainCompilador.java (Orquestrador Parte 1)

Como o ponto de entrada integra as 4 fases da compilação e captura erros de cada etapa

Dentro de src/compilador/MainCompilador.java (Linhas 40-82)

```
public class MainCompilador {
    public static void main(String[] args) {
        String caminhoFonte = "exemplos/correto.java.txt";
        String caminhoObjeto = "codigo.objeto.txt";
        boolean modoDebug = false;
        // ... lê args e flag --debug ...
        try {
            // FASE 1: ANÁLISE LÉXICA (Texto -> Lista de Tokens)
            Lexer analisadorLexico = Lexer.fromFile(caminhoFonte);
            List<Lexer.Token> listaDeTokens = analisadorLexico.tokenizarTodos();
            // FASE 2: ANÁLISE SINTÁTICA ASCENDENTE SLR(1)
            ParserSLR analisadorSintatico = new ParserSLR();
            analisadorSintatico.setModoDepuracao(modoDebug);
            ParserSLR.ResultadoParser resultadoSintatico = analisadorSintatico.analisar(listaDeToke
            // FASE 3: ANÁLISE SEMÂNTICA
            AnalisadorSemantico analisadorSemantico = new AnalisadorSemantico();
            AnalisadorSemantico.TabelaSimbolos tabelaSimbolos = analisadorSemantico.analisar(result
            tabelaSimbolos.imprimirTabela();
            // FASE 4: GERAÇÃO DE CÓDIGO OBJETO
            TradutorCodigo tradutor = new TradutorCodigo(tabelaSimbolos);
            tradutor.traduzir(resultadoSintatico.raiz);
            tradutor.salvarArquivo(caminhoObjeto);
            tradutor.imprimirCodigo();
```

Destaques de MainCompilador.java

Pipeline Limpo em 4 Estágios

Cada fase recebe a saída exata da fase anterior: Texto -> Tokens -> Árvore CST -> Tabela de Símbolos -> Código Objeto.

Hierarquia de Tratamento de Erros

Três blocos catch específicos diferenciam claramente se o código falhou no Léxico, no Sintático ou no Semântico.

Interrupção Segura

Se houver qualquer erro nas fases 1, 2 ou 3, o compilador aborta com System.exit(1) e NÃO gera código objeto inválido.

Argumentos Flexíveis

Pode rodar sem argumentos (usa correto.java.txt por padrão) ou passando qualquer arquivo de teste e '--debug'.

21. Arquivos 7 e 8: MaquinaHipotetica.java e MainMaquina.java

O Interpretador da CPU Virtual de Pilha (Parte 2): Registradores i e s, Memória M[] e Ciclo de Execução

Dentro de src/compilador/maquina/MaquinaHipotetica.java

```
private final List<InstrucaoVM> programa = new ArrayList<>();
private final double[] M = new double[10000]; // Memória + Pilha unificada
private int i = 0;  // Linha da próxima instrução
private int s = -1; // Topo da pilha de dados M[]

while (rodando && i >= 0 && i < programa.size()) {
    int linhaAtual = i++; // Lê a linha atual e já avança 'i' para a próxima instrução
    InstrucaoVM inst = programa.get(linhaAtual);
    int argInt = (inst.argumento != null) ? inst.argumento.intValue() : 0;
    switch (inst.operacao) {
        case "INPP": s = -1; break;
        case "ALME": s += argInt; break;
        case "DESM": s -= argInt; break;
        case "PARA": rodando = false; break;
        case "CRCT": s++; M[s] = inst.argumento; break;
        case "CRVL": s++; M[s] = M[argInt]; break;
        case "ARMZ": M[argInt] = M[s]; s--; break;
        case "SOMA": M[s - 1] = M[s - 1] + M[s]; s--; break;
        case "SUBT": M[s - 1] = M[s - 1] - M[s]; s--; break;
        case "DIVI": if (M[s] == 0.0) throw new ArithmeticException(...);
                     M[s - 1] = M[s - 1] / M[s]; s--; break;
        case "DSVI": i = argInt; break;
        case "DSVF": if (M[s] == 0.0) i = argInt; s--; break;
```

Detalhes Vitais de MaquinaHipotetica.java

Por que s começa em -1?

Porque o array M[] é 0-indexado. Quando a pilha está vazia, s = -1. No 1º empilhamento (s++; M[s] = ...), s vai para 0 (M[0]).

Como variáveis e pilha dividem M[]?

ALME 4 faz s subir de -1 para 3, reservando M[0..3] para variáveis fixas. Os cálculos empilham temporários de M[4] em diante!

Limpeza no carregarArquivo (48-52)

O leitor remove comentários '#' e prefixos '0:' automaticamente, aceitando qualquer arquivo de código objeto da disciplina.

Proteção em DIVI (Linha 101)

Verifica se o divisor no topo (M[s]) é 0.0 antes de dividir, lançando erro claro em tempo de execução.

22. Rastreamento Completo: correto.java.txt -> 36 Instruções

Veja como cada bloco do programa de teste vira instruções na Máquina Hipotética

Tradução Lado a Lado: Java -> Código Objeto

```
double cont; double a,b,c;  -->  0: INPP        |  1: ALME 4
cont = 10;                  -->  2: CRCT 10     |  3: ARMZ 0
while (cont > 0) {          -->  4: CRVL 0      |  5: CRCT 0
                                 6: CPMA        |  7: DSVF 32
    a = lerDouble();        -->  8: LEIT        |  9: ARMZ 1
    b = lerDouble();        --> 10: LEIT        | 11: ARMZ 2
    if (a > b) {            --> 12: CRVL 1      | 13: CRVL 2
                                14: CPMA        | 15: DSVF 21
        c = a - b;          --> 16: CRVL 1      | 17: CRVL 2
                                18: SUBT        | 19: ARMZ 3
    } else {                --> 20: DSVI 25 (pula o else!)
        c = b - a;          --> 21: CRVL 2      | 22: CRVL 1
                                23: SUBT        | 24: ARMZ 3
    }
    System.out.println(c);  --> 25: CRVL 3      | 26: IMPR
    cont = cont - 1;        --> 27: CRVL 0      | 28: CRCT 1
                                29: SUBT        | 30: ARMZ 0
}                           --> 31: DSVI 4  (volta ao while!)
System.out.println(c);      --> 32: CRVL 3      | 33: IMPR
} }                         --> 34: DESM 4      | 35: PARA
```

Mapa da Memória M[] Durante a Execução

M[0] = cont (Contador do laço)
Começa em 10.0 e é decrementado nas linhas 27..30 até chegar a 0.0.

M[1] = a e M[2] = b
Recebem os valores digitados pelo usuário nas instruções LEIT (linhas 8 e 10).

M[3] = c (Resultado)
Guarda a diferença absoluta |a - b| calculada pelo ramo if (16..19) ou else (21..24).

M[4] e M[5] (Pilha de Operandos)
Usados como rascunho temporário para empilhar operandos nas contas e comparações!

Equilíbrio Perfeito de Pilha
Ao fim de cada comando (;), o topo 's' volta sempre para 3. E no DESM 4 final, volta para -1!

23. Perguntas do Professor: Léxico e Sintático SLR(1)

Respostas diretas para 'O que essa linha faz?' em Lexer.java, TabelaSLR.java e ParserSLR.java

Guia Rápido: 'O Que Essa Linha Faz?' (Parte 1)

```
// [PERGUNTA 1] Lexer.java:143
if ("System".equals(palavra) && codigoFonte.startsWith(".out.println", posicaoAtual))
-> Reconhece 'System.out.println' como um único token atômico (Tipo.PRINT).

// [PERGUNTA 2] Lexer.java:214
private boolean consumirSeProximoFor(char esperado)
-> Lookahead condicional de 1 caractere para diferenciar '=' de '==', '<' de '<=', etc.

// [PERGUNTA 3] TabelaSLR.java:109
for (Lexer.Tipo tokenDoFollow : conjuntoFollow.get(regra.ladoEsquerdo))
-> Regra central do SLR(1): aplica REDUCE apenas nas colunas do FOLLOW(ladoEsquerdo).

// [PERGUNTA 4] ParserSLR.java:125-128
for (int k = 0; k < regra.ladoDireito.length; k++) {
    pilhaDeEstados.pop();
    novoNoPai.filhos.add(0, pilhaDeNos.pop());
}
-> Desempilha |ladoDireito| itens de cada pilha e corrige a ordem LIFO inserindo em 0.
```

Outras Perguntas Teóricas Frequentes

"Por que a gramática não tem Dangling Else?"
Porque as chaves '{ <CMDS> }' são obrigatórias no if e no else. O '}' fecha o bloco sem ambiguidade.

"Por que lerDouble() não está em FATOR?"
Para proibir leituras no meio de expressões (ex: 'a = lerDouble() + 1'). Só pode em 'id = lerDouble();'.

"Como o parser descobre os tokens esperados no erro?"
Em obterTokensEsperados(estadoAtual), retorna tabelaAction.get(estadoAtual).keySet(), listando as colunas válidas!

24. Perguntas do Professor: Semântico, Tradutor e Máquina VM

Respostas diretas para 'O que essa linha faz?' em AnalisadorSemantico, TradutorCodigo e MaquinaHipotetica

Guia Rápido: 'O Que Essa Linha Faz?' (Parte 2)

```
// [PERGUNTA 5] AnalisadorSemantico.java:51
Simbolo novo = new Simbolo(nome, "double", variaveisDeclaradas.size(), ...);
-> Atribui endereço sequencial (0, 1, 2...) correspondente à posição em M[].

// [PERGUNTA 6] AnalisadorSemantico.java:92
percorrerComandos(noRaizProg.getFilho(15));
-> Acessa o nó <CMDS> do método main (16º símbolo da regra R1: PROG).

// [PERGUNTA 7] TradutorCodigo.java:124 e 130
int linhaDsvf = emitir("DSVF", -1); ... corrigirDesvio(linhaDsvf, listaInstrucoes.size());
-> Técnica de Backpatching: emite salto com -1 e preenche o endereço real ao fechar o bloco.

// [PERGUNTA 8] MaquinaHipotetica.java:98 e 116
case "SUBT": M[s - 1] = M[s - 1] - M[s]; s--; break;
case "DSVF": if (M[s] == 0.0) i = argInt; s--; break;
-> SUBT subtrai o topo (2º operando) do sub-topo (1º operando). DSVF desempilha a condição.
```

Resumo Final para Tirar Nota Máxima

Por que int linhaAtual = i++? (Linha 75 da VM)

Avança o Program Counter 'i' antes de executar; se for DSVI/DSVF, o salto sobrescreve 'i'.

Diferença entre CRCT e CRVL

CRCT empilha constante imediata (ex: CRCT 10 -> 10.0); CRVL copia o valor da memória M[argInt] para o topo.

Diferença entre ALME e ARMZ

ALME n reserva 'n' espaços subindo 's += argInt'; ARMZ end desempilha o topo e salva na variável 'M[argInt] = M[s]; s--'.

Você está 100% preparado!

Assista quantas vezes quiser e use também a Apresentação de Slides para explorar cada estado e arquivo!